

Initialized Terraform:

```
akashmane@Akashs-MacBook-Air aws-project-2 % terraform init
Initializing modules...
[- api_gateway_lambda in modules/api_gateway_lambda
Initializing provider plugins found in the configuration...
- Finding hashicorp/archive versions matching "~> 2.4"...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
- Installing hashicorp/archive v2.8.0...
- Installed hashicorp/archive v2.8.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
akashmane@Akashs-MacBook-Air aws-project-2 % terraform validate
Success! The configuration is valid.
```

After terraform apply:

```
api_gateway_url = "https://5sbhamfznj.execute-api.us-east-1.amazonaws.com/dev/hello"
lambda_function_name = "aws-sensor-project-dev-hello-lambda"
```

After passing the api gateway url:

```
{
  "message": "Hello from AWS Lambda",
  "project": "aws-sensor-project"
}
```

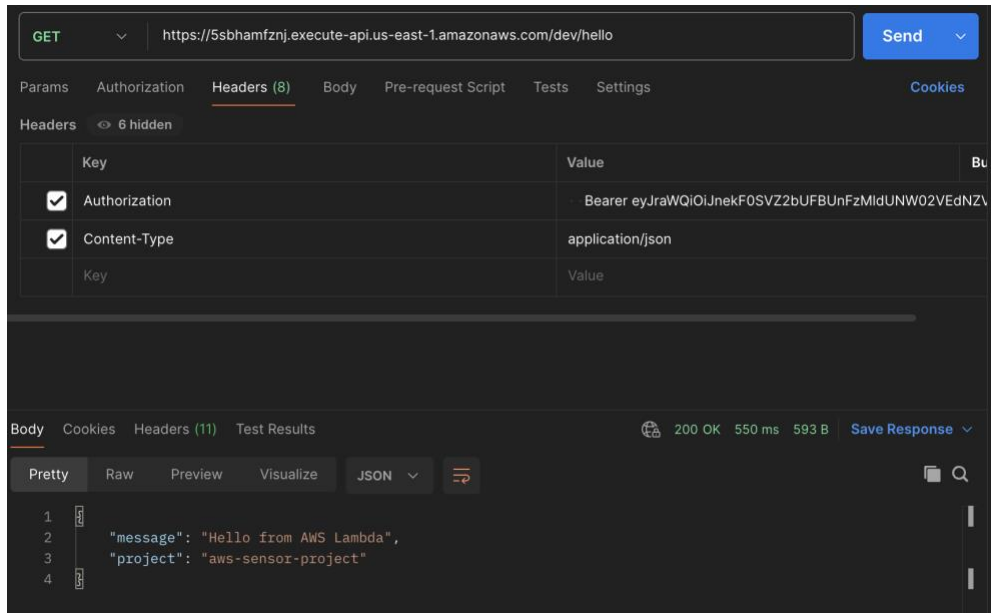
After adding AWS Cognito using terraform and entering terraform apply:

```
Apply complete! Resources: 4 added, 2 changed, 1 destroyed.

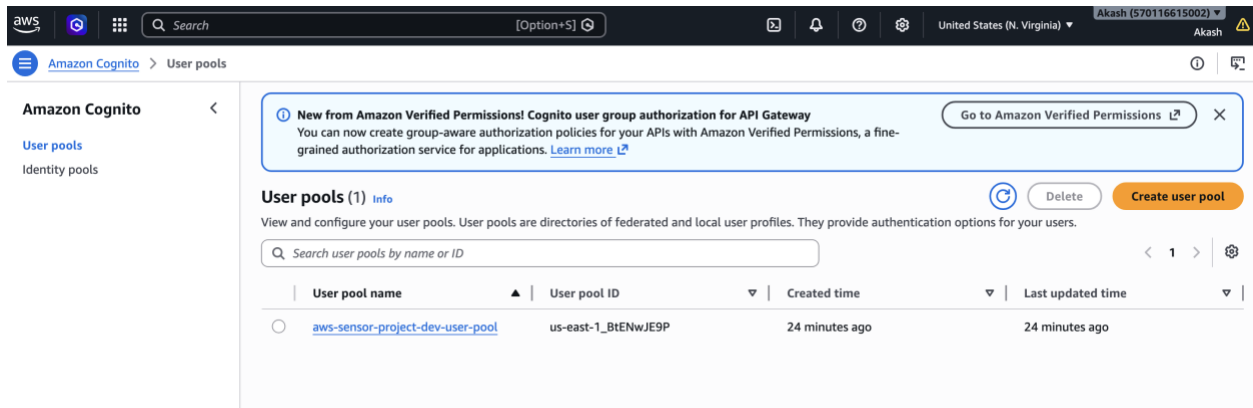
Outputs:

api_gateway_url = "https://5sbhamfznj.execute-api.us-east-1.amazonaws.com/dev/hello"
cognito_app_client_id = "6cofmj9j2r5quqn12kmdi05f0"
cognito_user_pool_id = "us-east-1_BtENwJE9P"
lambda_function_name = "aws-sensor-project-dev-hello-lambda"
```

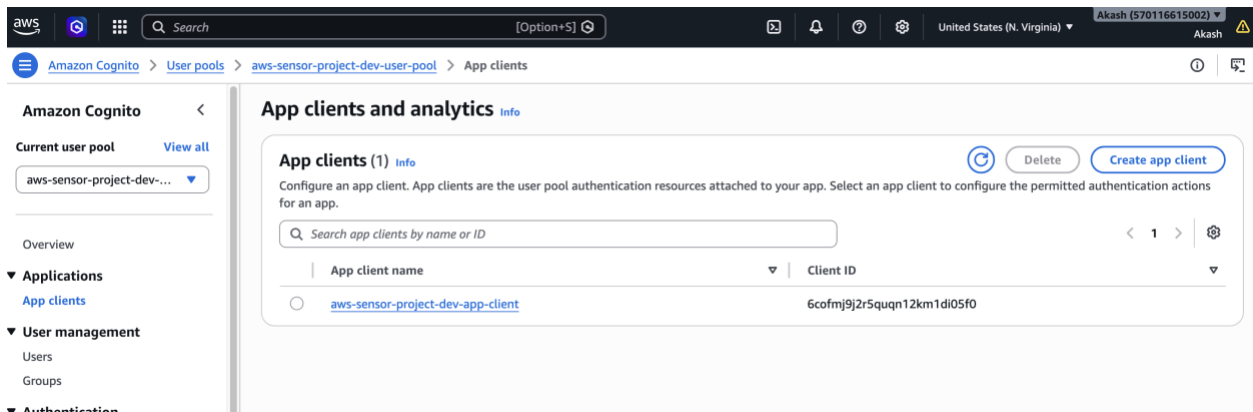
Accessing URL through POSTMAN by passing token in header:



User Pool:



App Client:



User:

The screenshot shows the Amazon Cognito console. The left sidebar has a menu with 'Amazon Cognito', 'Current user pool' (aws-sensor-project-dev-...), 'Overview', 'Applications' (App clients), 'User management' (Users, Groups), and 'Authentication'. The main content area is titled 'Users' and shows 'Users (1)'. It includes a search bar for 'Search users by attribute' and a table with columns: User name, Email address, Email verified, Confirmation status, and Status. One user is listed with email 'testuser@example.com' and status 'Enabled'. Below the table is the 'Import users (0)' section with a search bar and buttons for 'Start', 'Stop', and 'Create import job'.

API Gateway:

Cognito Authorizer

The screenshot shows the Amazon API Gateway console. The left sidebar has a menu with 'API Gateway', 'APIs', 'Custom domain names', 'Domain name access associations', 'VPC links', 'AgentCore targets', and 'API: aws-sensor-project-dev-api' (Resources, Stages, Authorizers, Gateway responses). The main content area is titled 'Authorizers' and shows 'Authorizers (1)'. It includes buttons for 'Edit', 'Delete', and 'Create authorizer'. One authorizer is listed: 'aws-sensor-project-dev-cognito-authorizer' with ID 'w4pubb', Cognito pool 'aws-sensor-project-dev-user-pool - BTENwJE9P (us-east-1)', Token source 'Authorization', and Token validation - optional 'none'.

The screenshot shows the Amazon API Gateway console. The left sidebar is the same as the previous screenshot. The main content area is titled 'Resources' and shows a tree view with a root resource '/' and a child resource '/hello' with a 'GET' method. To the right, the 'Resource details' section shows the path '/hello' and resource ID 'd2pmyu'. Below that, the 'Methods (1)' section shows a table with columns: Method type, Integration type, Authorization, and API key. One method is listed: 'GET' with integration type 'Lambda', authorization 'Cognito user pools', and API key 'Not required'.

After adding VPC, subnets and gateways:

```
Apply complete! Resources: 21 added, 0 changed, 0 destroyed.
```

Outputs:

```
alb_security_group_id = "sg-084f1c6daeebad392"
api_gateway_url = "https://5sbhamfznj.execute-api.us-east-1.amazonaws.com/dev/hello"
cognito_app_client_id = "6cofmj9j2r5uqn12kmdi05f0"
cognito_user_pool_id = "us-east-1_BtENwJE9P"
ec2_security_group_id = "sg-00122f3f74dc026b5"
lambda_function_name = "aws-sensor-project-dev-hello-lambda"
private_subnet_ids = [
    "subnet-0cbc7f3add9132b3e",
    "subnet-07036d41f7438e2e1",
]
public_subnet_ids = [
    "subnet-0c0537d6330bbe3c",
    "subnet-07d668ff0b6aa303c",
]
vpc_id = "vpc-0c6e694ecad1ee0c4"
```

VPC:

The screenshot displays the AWS Management Console interface for the VPC dashboard. The top navigation bar includes the AWS logo, a search bar, and the region 'United States (N. Virginia)'. The main header shows the breadcrumb 'VPCs > Your VPCs'. The left sidebar contains the 'VPC dashboard' and 'Virtual private cloud' sections. The main content area is titled 'Your VPCs' and includes a search bar with the placeholder 'Find VPCs by attribute or tag'. Below the search bar is a table of VPCs. The table has columns for Name, VPC ID, State, Encryption c..., Encryption control..., and Block Public... The first row shows a VPC named 'aws-sensor-project-dev-vpc' with ID 'vpc-0c6e694ecad1ee0c4' and state 'Available'. The table also includes a 'Filter by VPC' dropdown and a 'Create VPC' button.

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...
aws-sensor-project-dev-vpc	vpc-0c6e694ecad1ee0c4	Available	-	-	Off

Subnet:

Subnets (4) Info

Last updated 3 minutes ago [Actions](#) [Create subnet](#)

Find subnets by attribute or tag

VPC: vpc-0c6e694ecad1ee0c4 [Clear filters](#)

☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 C
☐	aws-sensor-project-dev-private-subnet-1	subnet-0cbcc7f3add9132b3e	Available	vpc-0c6e694ecad1ee0c4 aws-...	Off	10.0.1
☐	aws-sensor-project-dev-private-subnet-2	subnet-07036d41f7438e2e1	Available	vpc-0c6e694ecad1ee0c4 aws-...	Off	10.0.1
☐	aws-sensor-project-dev-public-subnet-2	subnet-07d668ff0b6aa303c	Available	vpc-0c6e694ecad1ee0c4 aws-...	Off	10.0.2
☐	aws-sensor-project-dev-public-subnet-1	subnet-0c0537d6330bbe3c	Available	vpc-0c6e694ecad1ee0c4 aws-...	Off	10.0.1

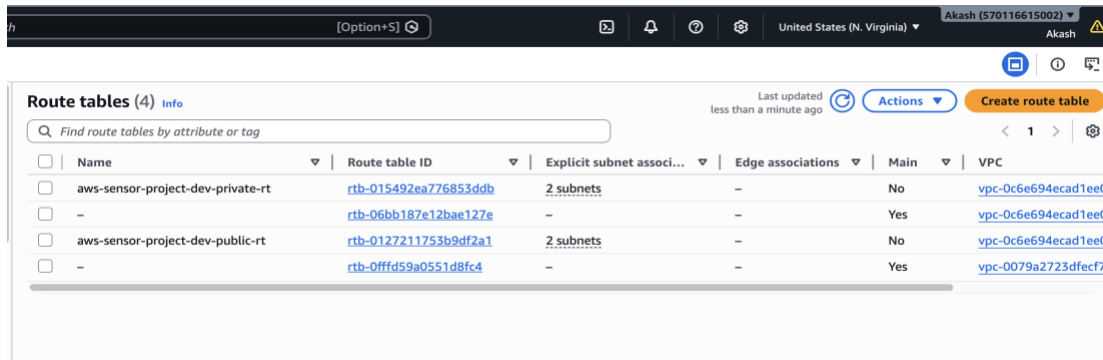
Internet Gateway:

Internet gateways (2) Info

Find internet gateways by attribute or tag

	Name	Internet gateway ID	State	VPC ID	Owner
☐	-	igw-05f5d016f3ea934fe	Attached	vpc-0079a2723dfecf7e0	57011661500
☐	aws-sensor-project-dev-igw	igw-0cbb84feb53c975a7	Attached	vpc-0c6e694ecad1ee0c4 aws-sensor-p...	57011661500

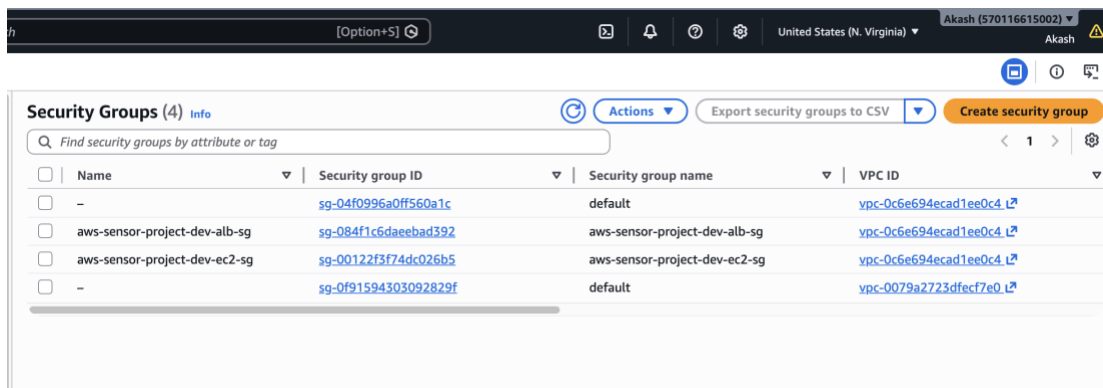
Route Table:



The screenshot shows the AWS Management Console interface for Route Tables in the United States (N. Virginia) region. The page title is "Route tables (4) Info". A search bar is present with the placeholder text "Find route tables by attribute or tag". The table lists four route tables, each associated with a VPC. The first two route tables are for the "aws-sensor-project-dev-private-rt" VPC, and the last two are for the "aws-sensor-project-dev-public-rt" VPC. The table columns are: Name, Route table ID, Explicit subnet associations, Edge associations, Main, and VPC.

Name	Route table ID	Explicit subnet associations	Edge associations	Main	VPC
aws-sensor-project-dev-private-rt	rtb-015492ea776853ddb	2 subnets	-	No	vpc-0c6e694ecad1ee0c4
-	rtb-06bb187e12bae127e	-	-	Yes	vpc-0c6e694ecad1ee0c4
aws-sensor-project-dev-public-rt	rtb-0127211753b9df2a1	2 subnets	-	No	vpc-0c6e694ecad1ee0c4
-	rtb-0fffd59a0551d8fc4	-	-	Yes	vpc-0079a2723dfecf7e0

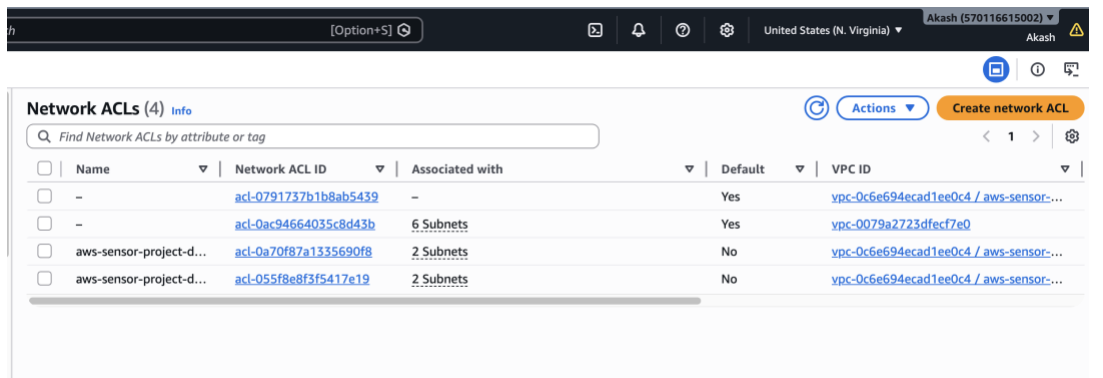
Security Groups:



The screenshot shows the AWS Management Console interface for Security Groups in the United States (N. Virginia) region. The page title is "Security Groups (4) Info". A search bar is present with the placeholder text "Find security groups by attribute or tag". The table lists four security groups, each associated with a VPC. The first two security groups are for the "aws-sensor-project-dev-alb-sg" VPC, and the last two are for the "aws-sensor-project-dev-ec2-sg" VPC. The table columns are: Name, Security group ID, Security group name, and VPC ID.

Name	Security group ID	Security group name	VPC ID
-	sg-04f0996a0ff560a1c	default	vpc-0c6e694ecad1ee0c4
aws-sensor-project-dev-alb-sg	sg-084f1c6daeebad392	aws-sensor-project-dev-alb-sg	vpc-0c6e694ecad1ee0c4
aws-sensor-project-dev-ec2-sg	sg-00122f3f74dc026b5	aws-sensor-project-dev-ec2-sg	vpc-0c6e694ecad1ee0c4
-	sg-0f91594303092829f	default	vpc-0079a2723dfecf7e0

NACL:



The screenshot shows the AWS Management Console interface for Network ACLs in the United States (N. Virginia) region. The page title is "Network ACLs (4) Info". A search bar is present with the placeholder text "Find Network ACLs by attribute or tag". The table lists four network ACLs, each associated with a VPC. The first two network ACLs are for the "aws-sensor-project-dev-private-rt" VPC, and the last two are for the "aws-sensor-project-dev-public-rt" VPC. The table columns are: Name, Network ACL ID, Associated with, Default, and VPC ID.

Name	Network ACL ID	Associated with	Default	VPC ID
-	acl-0791737b1b8ab5439	-	Yes	vpc-0c6e694ecad1ee0c4 / aws-sensor-project-dev-private-rt
-	acl-0ac94664035c8d43b	6 Subnets	Yes	vpc-0079a2723dfecf7e0
aws-sensor-project-dev-private-rt	acl-0a70f87a1335690f8	2 Subnets	No	vpc-0c6e694ecad1ee0c4 / aws-sensor-project-dev-private-rt
aws-sensor-project-dev-public-rt	acl-055f8e8f3f5417e19	2 Subnets	No	vpc-0c6e694ecad1ee0c4 / aws-sensor-project-dev-public-rt

Upon adding Internal ALB and EC2 instances in terraform and entering terraform apply:

```
Outputs:

alb_security_group_id = "sg-084f1c6daeebad392"
api_gateway_url = "https://5sbhamfznj.execute-api.us-east-1.amazonaws.com/dev/hello"
cognito_app_client_id = "6cofmj9j2r5quqn12km1d105f0"
cognito_user_pool_id = "us-east-1_BtENwJE9P"
ec2_instance_id = "i-06c81df515b8e1af1"
ec2_security_group_id = "sg-00122f3f74dc026b5"
internal_alb_dns_name = "internal-sensor-dev-alb-1361902677.us-east-1.elb.amazonaws.com"
lambda_function_name = "aws-sensor-project-dev-hello-lambda"
private_subnet_ids = [
    "subnet-0cbc7f3add9132b3e",
    "subnet-07036d41f7438e2e1",
]
public_subnet_ids = [
    "subnet-0c0537d6330bbe3c",
    "subnet-07d668ff0b6aa303c",
]
target_group_name = "sensor-dev-tg"
vpc_id = "vpc-0c6e694ecad1ee0c4"
```

EC2 Instances:

The screenshot shows the AWS Management Console for the 'Instances' page. The left sidebar contains navigation links for 'Dashboard', 'AWS Global View', 'Events', 'Instances', and 'Instance Types'. The main content area shows 'Instances (1)' with a search bar and a table of instances. The table has columns for Name, Instance ID, Instance state, Instance type, Status check, and Availability Zone. One instance is listed: 'aws-sensor-project-dev-springboot-ec2' with ID 'i-06c81df515b8e1af1', state 'Running', type 't3.micro', and availability zone 'us-east-1a'.

EC2 Load Balancer:

The screenshot shows the AWS Management Console for the 'Load balancers' page. The left sidebar contains navigation links for 'Savings Plans', 'Reserved Instances', 'Dedicated Hosts', 'Capacity Reservations', 'Capacity Manager', 'Images', 'AMIs', and 'AMI Catalog'. The main content area shows 'Load balancers (1)' with a search bar and a table of load balancers. The table has columns for Name, State, Type, Scheme, IP address type, VPC ID, and Availability Zone. One load balancer is listed: 'sensor-dev-alb' with state 'Active', type 'application', scheme 'Internal', IP address type 'IPv4', VPC ID 'vpc-0c6e694ecad1ee0c4', and availability zone 'us-east-1a'.

EC2 Target Group:

The screenshot shows the AWS Management Console for the 'Target groups' page. The left sidebar contains navigation links for 'Savings Plans', 'Reserved Instances', 'Dedicated Hosts', 'Capacity Reservations', 'Capacity Manager', 'Images', 'AMIs', and 'AMI Catalog'. The main content area shows 'Target groups (1)' with a search bar and a table of target groups. The table has columns for Name, ARN, Port, Protocol, Target type, Load balancer, and Health status. One target group is listed: 'sensor-dev-tg' with ARN 'arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/sensor-dev-tg/123456789012', port '8080', protocol 'HTTP', target type 'Instance', load balancer 'sensor-dev-alb', and health status 'Healthy'.

EC2 Target Health (Healthy):

The screenshot shows the AWS Management Console for the 'Targets' page. The left sidebar contains navigation links for 'Targets', 'Monitoring', 'Health checks', 'Attributes', and 'Tags'. The main content area shows 'Registered targets (1)' with a search bar and a table of targets. The table has columns for Instance ID, Name, Port, Zone, Health status, and Health status details. One target is listed: 'aws-sensor-project-dev-springboot-ec2' with ID 'i-06c81df515b8e1af1', port '8080', zone 'us-east-1a (us-east-1a)', health status 'Healthy', and health status details '-'. The page also shows tabs for 'Targets', 'Monitoring', 'Health checks', 'Attributes', and 'Tags'.

After adding VPC link to the API Gateway and entering terraform apply:

```
alb_listener_arn = "arn:aws:elasticloadbalancing:us-east-1:570116615002:listener/app/sensor-dev-alb/fb0ff40fcf3fae3/0dbb70cf43a49944"
alb_security_group_id = "sg-084f1c6daeebad392"
api_gateway_url = "https://5sbhamfznj.execute-api.us-east-1.amazonaws.com/dev/hello"
backend_api_url = "https://ll6ho2hy3j.execute-api.us-east-1.amazonaws.com/backend"
cognito_app_client_id = "6cofmj9j2r5quqn12km1di05f0"
cognito_user_pool_id = "us-east-1_BtENwJE9P"
ec2_instance_id = "i-06c81df515b8e1af1"
ec2_security_group_id = "sg-00122f3f74dc026b5"
internal_alb_dns_name = "internal-sensor-dev-alb-1361902677.us-east-1.elb.amazonaws.com"
lambda_function_name = "aws-sensor-project-dev-hello-lambda"
private_subnet_ids = [
  "subnet-0cbc7f3add9132b3e",
  "subnet-07036d41f7438e2e1",
]
public_subnet_ids = [
  "subnet-0c0537d6330bbee3c",
  "subnet-07d668ff0b6aa303c",
]
target_group_name = "sensor-dev-tg"
vpc_id = "vpc-0c6e694ecad1ee0c4"
vpc_link_id = "0x1nv0"
```

After entering this API gateway URL:

<https://ll6ho2hy3j.execute-api.us-east-1.amazonaws.com/backend>

```
Pretty-print ☒
{
  "message": "Hello from EC2 backend",
  "service": "springboot-file-upload-placeholder"
}
```

Before deploying the jar file on EC2, testing the url locally:

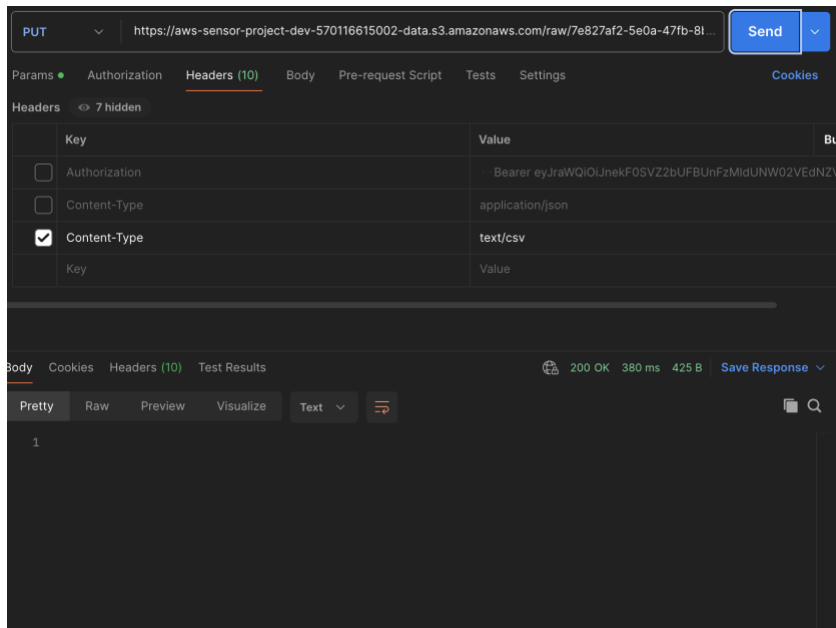
The screenshot shows a REST client interface. The top bar indicates a POST request to `http://localhost:8080/backend/files/presigned-url`. The 'Body' tab is selected, showing a JSON payload:

```
{
  "fileName": "sensor-data.csv",
  "contentType": "text/csv"
}
```

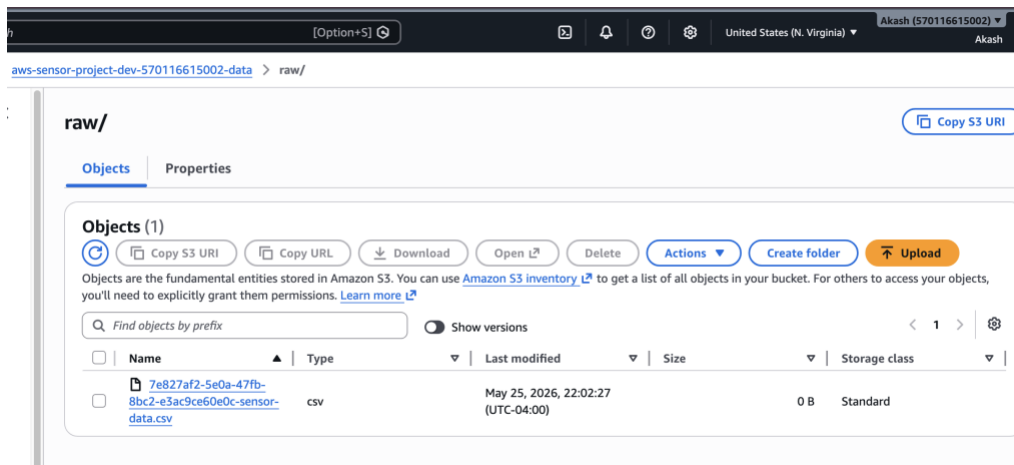
Below the request, the response is displayed. The status is 200 OK, with a response time of 293 ms and a body size of 706 B. The response is shown in the 'Pretty' tab as a JSON object:

```
{
  "uploadUrl": "https://aws-sensor-project-dev-570116615002-data.s3.amazonaws.com/raw/7e827af2-5e0a-47fb-8bc2-e3ac9ce60e0c-sensor-data.csv?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Date=20260526T015755Z&X-Amz-SignedHeaders=content-type%3Bhost%3Bx-amz-credential%3D%2F20260526%2Fus-east-1%2Faws4_request&X-Amz-Expires=600&X-Amz-Signature=1da23ba0e882a68dae0166be3055ce1c4e3c261406ee8e0d8129addaf0684992",
  "s3Key": "raw/7e827af2-5e0a-47fb-8bc2-e3ac9ce60e0c-sensor-data.csv",
  "bucketName": "aws-sensor-project-dev-570116615002-data"
}
```

After passing the upload url via PUT,

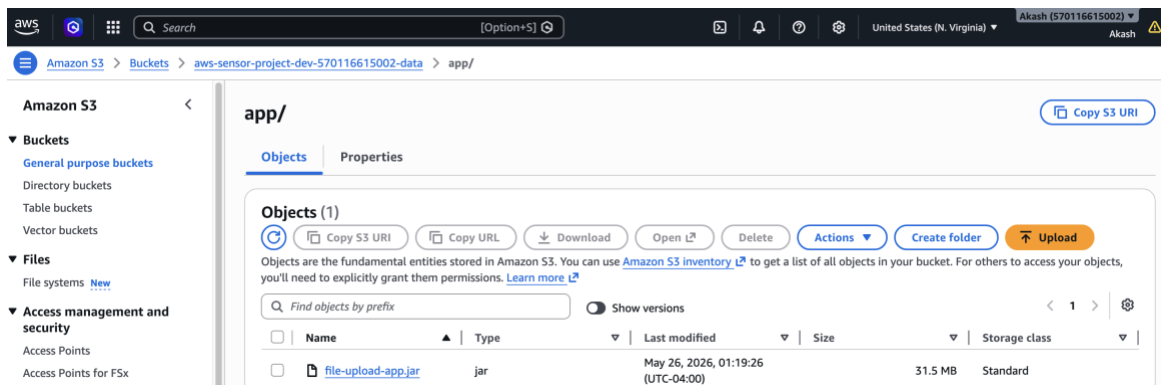


The .csv file is generated inside S3/bucket/raw/



After deploying jar on EC2,

S3 Bucket



EC2 instance in Target Group is healthy:

The screenshot shows the AWS Management Console for a Target Group named 'sensor-dev-tg'. The console displays the following information:

- Target type:** Instance
- Protocol : Port:** HTTP: 8080
- Protocol version:** HTTP1
- VPC:** vpc-0c6e694ecad1ee0c4
- IP address type:** IPv4
- Load balancer:** sensor-dev-alb

The 'Targets' tab shows a table with the following data:

Target ID	Health status	Unhealthy	Unused	Initial	Draining
1	Healthy	0	0	0	0

The 'Monitoring' tab shows the 'Registered targets' table with the following data:

Instance ID	Name	Port	Zone	Health status	Health status details	Admin...
i-079a39a7a3d1e54fd	aws-sensor-pr...	8080	us-east-1a (us...	Healthy	-	No override.

Passing presigned url and passing .csv in request body, it uploads the .csv file in /raw in S3

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** https://aws-sensor-project-dev-570116615002-data.s3.amazonaws.com/raw/fa46bbd6-1c71-4ab5-a...
- Headers:**
 - Authorization: Bearer eyJraWQOIjnekF0SVZ2bUFBUnFzMldUNW02VEdNZV...
 - Content-Type: application/json
 - Content-Type: text/csv
- Body:** (Empty)
- Status:** 200 OK, 257 ms, 445 B

In S3 bucket,

The screenshot shows the Amazon S3 console for the 'raw/' prefix. The console displays the following information:

- Prefix:** raw/
- Copy S3 URI:** (Button)
- Objects (2):**
 - Object 1:** 7e827af2-5e0a-47fb-8bc2-e3ac9ce60e0c-sensor-data.csv (CSV file, 0 B, Standard storage class, May 25, 2026, 22:02:27 UTC-04:00)
 - Object 2:** fa46bbd6-1c71-4ab5-abe6-211e973df714-sensor-data.csv (CSV file, 0 B, Standard storage class, May 26, 2026, 01:26:33 UTC-04:00)

After adding EventBridge and StepFunction to the S3 Bucket,

The screenshot shows the AWS Step Functions console. The left sidebar contains navigation links for Step Functions, State machines, and various developer resources. The main content area displays the details for the state machine 'aws-sensor-project-dev-sensor-pipeline'. The 'Details' tab is active, showing the ARN, IAM role ARN, Type (Standard), Status (Active), Creation date (May 26, 2026, 02:58:11 UTC-04:00), and X-Ray tracing (Disabled). Below the details, there are tabs for Executions, Monitoring, Logging, Definition, Aliases, Versions, and Tags. The 'Executions' tab is selected, showing a table of executions. One execution is listed with ID '99f8d539-121d-f6a8-a0c5-104a66c92342', status 'Succeeded', and a duration of 00:00:00.127.

aws-sensor-project-dev-sensor-pipeline

Details

Arn: [arn:aws:states:us-east-1:570116615002:stateMachine:aws-sensor-project-dev-sensor-pipeline](#)

IAM role ARN: [arn:aws:iam::570116615002:role/aws-sensor-project-dev-step-function-role](#)

Type: Standard

Status: Active

Creation date: May 26, 2026, 02:58:11 (UTC-04:00)

X-Ray tracing: Disabled

Executions (0/1)

Name	Status	Start Time (local)	End Time (local)	Duration	Info	Version
99f8d539-121d-f6a8-a0c5-104a66c92342...	Succeeded	May 26, 2026, 03:39:29	May 26, 2026, 03:39:29	00:00:00.127		-

The screenshot shows the AWS Step Functions console, specifically the execution details for the execution ID '99f8d539-121d-f6a8-a0c5-104a66c92342_2d917069-0c06-60...'. The 'Graph view' is selected, showing a flow diagram with the following steps: Start -> Pass state (ReceiveS3UploadEvent) -> Succeed state (WorkflowCompleted) -> End. The 'Step details' panel on the right is empty, prompting the user to choose a step to view its details.

aws-sensor-project-dev-sensor-pipeline

Execution: 99f8d539-121d-f6a8-a0c5-104a66c92342_2d917069-0c06-60...

Graph view

```
graph TD; Start((Start)) --> PassState[Pass state  
ReceiveS3UploadEvent]; PassState --> SucceedState[Succeed state  
WorkflowCompleted]; SucceedState --> End((End));
```

Step details

Choose a step to view its details.

The screenshot shows the AWS EventBridge console. The left sidebar contains navigation links for Amazon EventBridge, Rules, and various developer resources. The main content area displays the details for the event rule 'aws-sensor-project-dev-s3-raw-upload-rule'. The 'Rule details' tab is active, showing the Rule name, Status (Enabled), Rule ARN, Event bus name (default), Event bus ARN, and Type (Standard). Below the details, there are tabs for Event pattern, Targets, Monitoring, and Tags. The 'Targets' tab is selected, showing a table of targets. One target is listed with Name 'aws-sensor-project-dev-sensor-pipeline', Type 'Step Functions state machine', and Role 'aws-sensor-project-dev-eventbridge-sfn-role'.

aws-sensor-project-dev-s3-raw-upload-rule

Rule details

Rule name: aws-sensor-project-dev-s3-raw-upload-rule

Status: Enabled

Rule ARN: [arn:aws:events:us-east-1:570116615002:rule/aws-sensor-project-dev-s3-raw-upload-rule](#)

Event bus name: default

Event bus ARN: [arn:aws:events:us-east-1:570116615002:event-bus/default](#)

Type: Standard

Targets

Details	Target Name	Type	ARN	Input	Role
	aws-sensor-project-dev-sensor-pipeline	Step Functions state machine	arn:aws:states:us-east-1:570116615002:stateMachine:aws-sensor-project-dev-sensor-pipeline	Matched event	aws-sensor-project-dev-eventbridge-sfn-role

Input to target: Matched event

Additional parameters: --

Ran Glue Job 1 manually,

The screenshot shows the AWS Glue console for a job named 'aws-sensor-project-dev-clean-csv-job'. The 'Runs' tab is selected, showing a table of job runs. The first run is 'Succeeded' with 0 retries, starting at 05/26/2026 17:00:45 and ending at 05/26/2026 17:01:47, with a duration of 55 seconds and 2 DPU capacity. The job was last updated on 5/26/2026 at 4:54:40 PM.

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity	Worker type
Succeeded	0	05/26/2026 17:00:45	05/26/2026 17:01:47	55 s	2 DPUs	G.1X

Cleaned CSV file stored in S3 bucket inside cleaned folder:

The screenshot shows the AWS S3 console for the bucket 'aws-sensor-project-dev-570116615002-data'. The 'cleaned/' folder is selected, showing a table of objects. The first object is a CSV file named 'part-00000-4cacfb5-36e0-4743-8990-e15825b09d1f-c000.csv', created on May 26, 2026, at 17:01:37 (UTC-04:00), with a size of 330.0 B and Standard storage class.

Name	Type	Last modified	Size	Storage class
part-00000-4cacfb5-36e0-4743-8990-e15825b09d1f-c000.csv	csv	May 26, 2026, 17:01:37 (UTC-04:00)	330.0 B	Standard

Ran Glue Job 2 manually,

The screenshot shows the AWS Glue console for a job named 'aws-sensor-project-dev-convert-to-parquet-job'. The 'Runs' tab is selected, showing a table of job runs. The first run is 'Succeeded' with 0 retries, starting at 05/26/2026 17:09:58 and ending at 05/26/2026 17:11:18, with a duration of 1 m 13 s and 2 DPU capacity. The job was last updated on May 26, 2026 at 21:11:34.

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity	Worker type
Succeeded	0	05/26/2026 17:09:58	05/26/2026 17:11:18	1 m 13 s	2 DPUs	G.1X

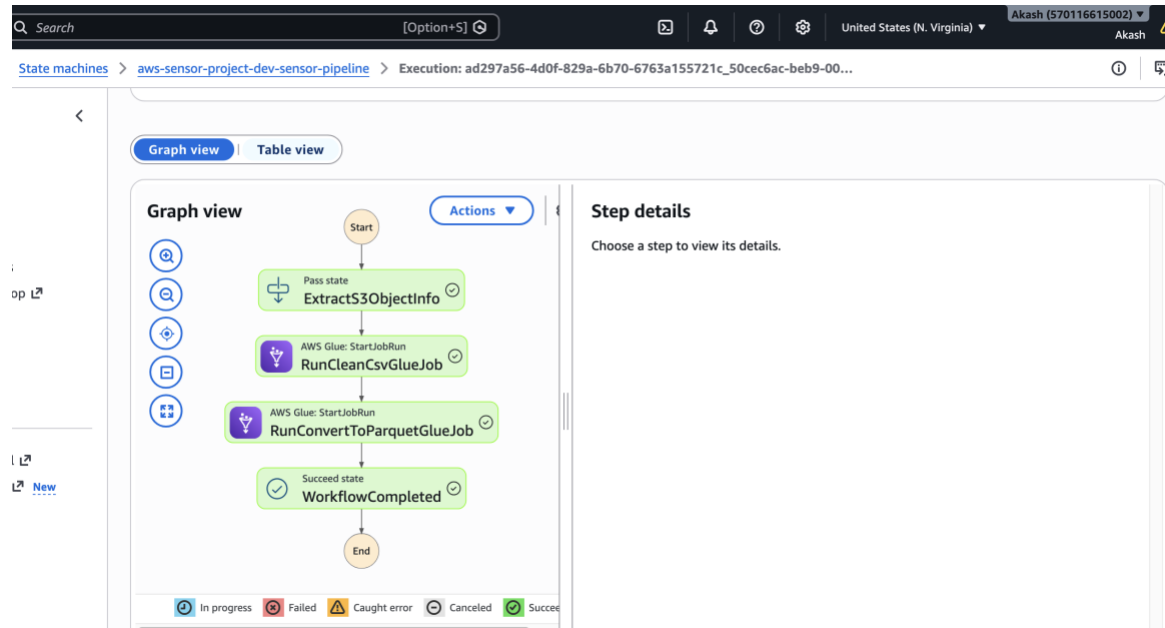
Cleaned CSV converted to Parquet file and stored inside parquet folder in S3 bucket

The screenshot shows the AWS S3 console for the bucket 'insor-project-dev-570116615002-data'. The 'parquet/' folder is selected, showing a table of objects. The first object is a Parquet file named 'part-00000-8a496273-9b12-44d1-be91-4f65aebd0b8a-c000.snappy.parquet', created on May 26, 2026, at 17:11:08 (UTC-04:00), with a size of 2.0 KB and Standard storage class.

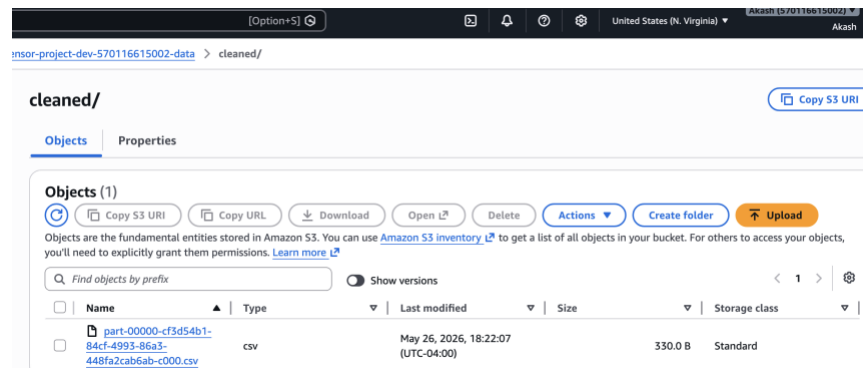
Name	Type	Last modified	Size	Storage class
part-00000-8a496273-9b12-44d1-be91-4f65aebd0b8a-c000.snappy.parquet	parquet	May 26, 2026, 17:11:08 (UTC-04:00)	2.0 KB	Standard

After connecting Glue Job 1 and Job 2, now Step Function automatically calls Job 1 and Job 2 once it gets object event from EventBridge

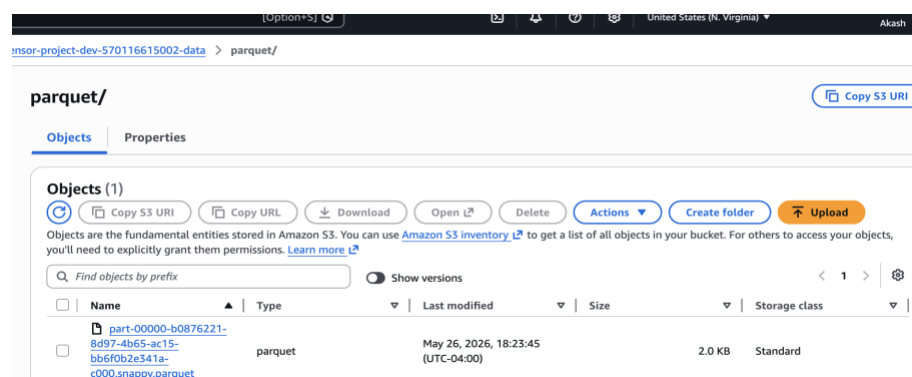
StateMachine:



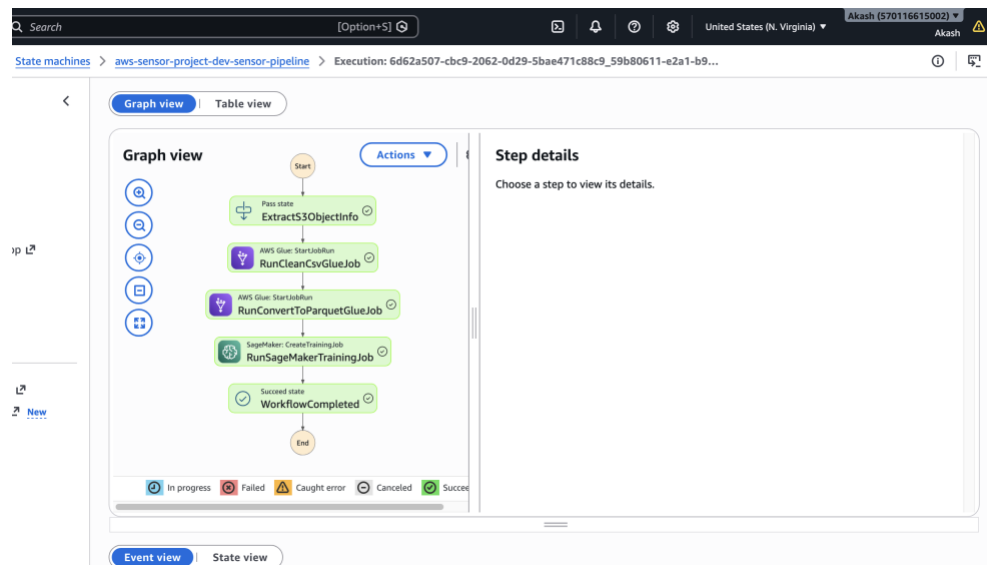
Cleaned CSV file under cleaned folder in S3:



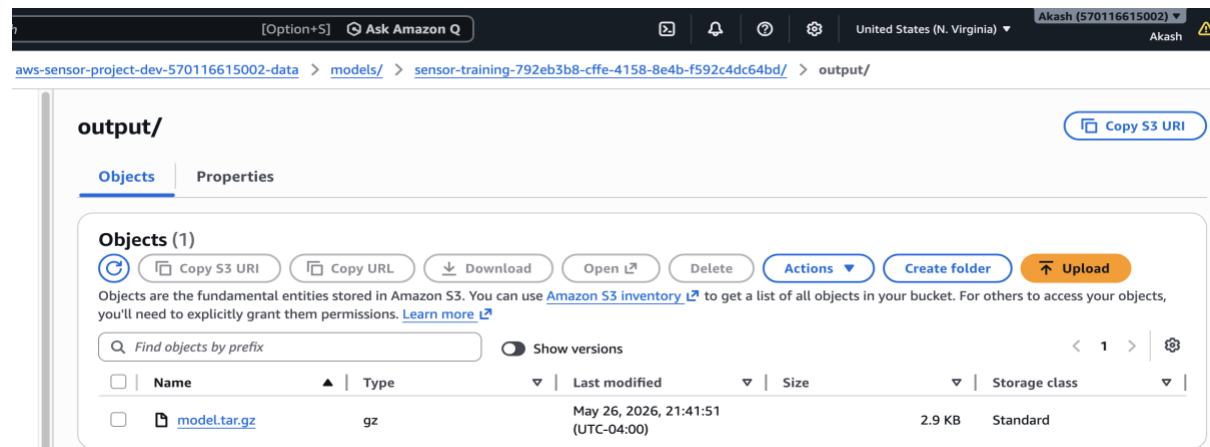
Cleaned CSV converted to Parquet and stored in the Parquet folder in S3



This is the Step Function which has now SageMaker in it. SageMaker will run the train.py which is converted to zip and it has the uploaded .csv file, with the help of these files, it will train the model which will be stored in the S3 bucket inside model folder



Model stored in bucket is in the .tar.gz format



After creating SageMaker endpoint,

The screenshot displays the AWS SageMaker console interface. The top navigation bar includes a search bar, a dropdown menu for 'United States (N. Virginia)', and a user profile 'Akash (570116615002)'. The main content area is titled 'Endpoints' and has a search bar for 'Find endpoints'. There are buttons for 'Update endpoint', 'Actions', and 'Create endpoint'. Below this, there is a table listing the endpoints.

	Name	ARN	Creation time	Status	Last updated
	aws-sensor-project-dev-sensor-endpoint	arn:aws:sagemaker:us-east-1:570116615002:endpoint/aws-sensor-project-dev-sensor-endpoint	5/27/2026, 1:18:13 AM	InService	5/27/2026, 1:23:59 AM

SageMaker endpoint configuration,

The screenshot shows the AWS SageMaker console interface for configuring an endpoint. The breadcrumb navigation indicates the path: Endpoint configuration > aws-sensor-project-dev-sensor-endpoint-config. The main title is 'aws-sensor-project-dev-sensor-endpoint-config', with buttons for 'Clone', 'Delete', and 'Apply to endpoint'.

Endpoint configuration settings

Name	ARN	Encryption key	Creation time
aws-sensor-project-dev-sensor-endpoint-config	arn:aws:sagemaker:us-east-1:570116615002:endpoint-config/aws-sensor-project-dev-sensor-endpoint-config	-	5/27/2026, 1:18:12 AM

Data capture

Enable data capture	Data capture options	S3 location to store data collected	Capture content type
No	-	-	-

Sampling percentage (%)

-

Variants

Identifies a model that you want to host and the resources chosen to deploy for hosting it.

Model name	Training job	Variant name	Instance type	Elastic Infer
aws-sensor-project-dev-sensor-model	sensor-training-26729e19-30c7-4e91-8b10-470d2e3892ab	AllTraffic	ml.t2.medium	-

Inference Output:

```
akashmane@Akashs-MacBook-Air aws-project-2 % aws sagemaker-runtime invoke-endpoint \
--endpoint-name aws-sensor-project-dev-sensor-endpoint \
--content-type text/csv \
--body "90.5,60.2,1010.1" \
--cli-binary-format raw-in-base64-out \
[ prediction-output.json
{
  "ContentType": "text/html; charset=utf-8",
  "InvokedProductionVariant": "AllTraffic"
}
[akashmane@Akashs-MacBook-Air aws-project-2 % cat prediction-output.json
critical%
```